

**Universidad Católica Nuestra Señora de
la Asunción**

Facultad de Ciencias y Tecnología

Microprocesadores II

Trabajo Práctico

Juego Snake en Zybo

Profesor: Vicente González

Alumna: Tania Patricia Romero Centurión

Matrícula: Y06343

Curso: 5to

Asunción - Paraguay

2025

Contenido

1	Introducción	2
2	Capítulo 1: Especificación del Sistema	3
2.1	Objetivo general	3
2.2	Entradas del sistema	4
2.3	Lógica de lectura y prioridades	4
2.4	Comunicación con el sistema VGA	4
2.5	Comportamiento ante eventos	5
3	Capítulo 2: Análisis	6
3.1	Descripción del problema	6
3.2	Entorno de desarrollo	6
3.3	Arquitectura general del sistema	6
3.4	Análisis funcional	7
3.5	Riesgos técnicos identificados	7
4	Capítulo 3: Diseño	8
4.1	Diagrama de bloques del sistema	8
4.2	Módulos VHDL diseñados	8
4.3	Programa en MIPS	9
4.4	Comunicación entre hardware y software	10
5	Capítulo 4: Implementación	11
5.1	Flujo general de ejecución	11
5.2	Conexión entre módulos	11
5.3	Código MIPS y control del juego	12
5.4	Generación de fruta y lógica caótica	12
6	Capítulo 5: Pruebas y Simulación	13
6.1	Pruebas en simulación VHDL	13
6.2	Pruebas funcionales en hardware	13
6.3	Análisis de resultados	14
6.4	Limitaciones y posibles mejoras	14
7	Conclusión	15
8	Anexos	17

1 Introducción

El presente trabajo práctico tiene como objetivo desarrollar el clásico juego de la serpiente (*Snake*) utilizando una plataforma FPGA Zybo (basada en el chip Zynq-7000) como entorno de implementación. El sistema combina el uso de hardware descrito en VHDL para la generación de la señal VGA, con un programa en lenguaje ensamblador MIPS encargado de controlar la lógica del juego.

La serpiente es representada gráficamente en una pantalla mediante salida VGA, con una resolución adaptada a una grilla lógica para simplificar el diseño y reducir el uso de recursos. El movimiento de la cabeza de la serpiente se controla desde el software MIPS, el cual se comunica con los módulos de hardware a través de registros mapeados en memoria. Estos registros permiten definir la posición de la cabeza, actualizar la dirección, y detectar eventos como la colisión con la fruta o con el cuerpo de la propia serpiente.

Además del movimiento, se implementa la lógica de crecimiento al comer la fruta, detección de colisiones y reinicio del juego. Para introducir elementos de aleatoriedad en la posición de la fruta, se utilizan contadores manipulados por eventos externos como pulsaciones de botones.

Este trabajo integra conocimientos clave del curso de Microprocesadores 2, tales como, control de periféricos mediante memoria mapeada, diseño digital en VHDL, y sincronización entre software y hardware en sistemas embebidos.

2 Capítulo 1: Especificación del Sistema

2.1 Objetivo general

Desarrollar un videojuego funcional del tipo *Snake* implementado sobre una plataforma FPGA Zybo, utilizando el procesador MIPS como unidad de control, y una interfaz de video VGA para la visualización. El sistema debe responder a entradas del usuario, gestionar colisiones y mantener el estado del juego de forma autónoma.

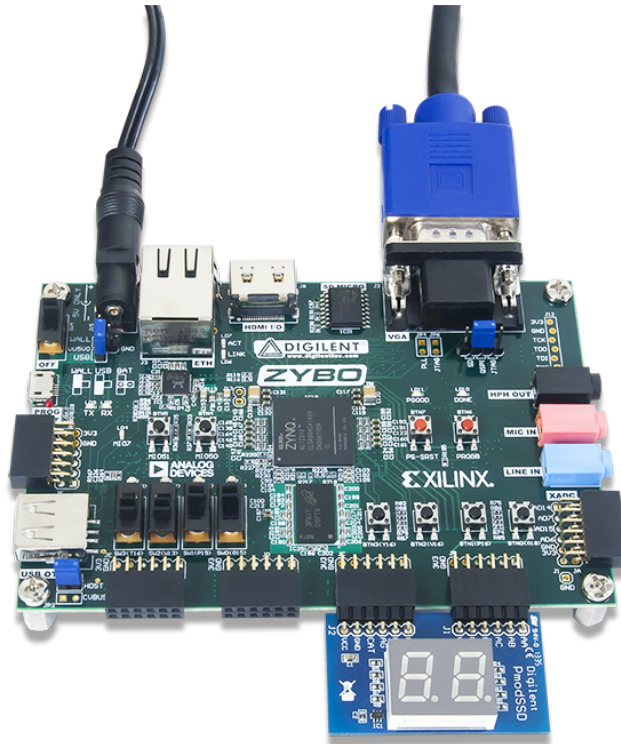


Figure 1: Zybo

2.2 Entradas del sistema

El sistema cuenta con dos tipos de entradas físicas provenientes del usuario:

- **Botones (direccionales):** permiten modificar la dirección de la serpiente en las cuatro direcciones cardinales (arriba, abajo, izquierda y derecha).
- **Switches:** utilizados para controlar el estado global del juego. Un switch activa el modo **pausa**, y otro activa el **reinicio** del sistema.

2.3 Lógica de lectura y prioridades

Las entradas se gestionan en el software MIPS a través de lectura de un registro mapeado en memoria que contiene el estado actual de los botones y switches. Se aplica un esquema jerárquico de prioridades para su procesamiento:

1. **Mayor prioridad: Switch de reinicio.** Al ser detectado, provoca el reinicio inmediato del sistema. El procesador MIPS envía las señales correspondientes a los módulos VGA para reiniciar la posición de la cabeza, resetear la longitud de la cola, reiniciar el estado de las banderas y generar una nueva fruta.
2. **Segunda prioridad: Switch de pausa.** Detiene temporalmente la actualización del juego. Mientras la pausa esté activa, se ignoran los botones direccionales y el juego se congela.
3. **Menor prioridad: Botones direccionales.** Si no hay ninguna señal de pausa o reinicio, se analiza el estado de los botones para actualizar la dirección de la cabeza de la serpiente. El nuevo valor se escribe en un registro dedicado, que luego es leído por el hardware VGA para actualizar el movimiento.

2.4 Comunicación con el sistema VGA

La interacción entre el procesador MIPS y los módulos de video se da mediante registros mapeados en memoria. El procesador escribe:

- La nueva dirección de movimiento.
- La posición de la cabeza.
- La longitud de la cola.

- Banderas de control (pausa, reset, fruta confirmada).

El módulo VGA, por su parte, realiza las siguientes tareas:

- Mueve la cola de la serpiente en función de la nueva dirección de la cabeza.
- Detecta colisiones entre la cabeza y la fruta.
- Detecta colisiones entre la cabeza y la propia cola.
- Genera las banderas correspondientes que se envían al software:
 - `FLAG_COLISION_FRUTA`: al detectar que la cabeza tocó la fruta.
 - `FLAG_COLISION_COLA`: al detectar una autocolisión.

2.5 Comportamiento ante eventos

- **Colisión con fruta:** el procesador MIPS, al detectar la bandera correspondiente, incrementa la longitud de la cola y la puntuación.
- **Colisión con la cola:** al recibir la señal de autocolisión, el software activa automáticamente el reinicio del juego, restaurando el estado inicial del sistema.

3 Capítulo 2: Análisis

3.1 Descripción del problema

El objetivo del proyecto es implementar un videojuego funcional del tipo *Snake* en una plataforma FPGA Zybo, utilizando una arquitectura compuesta por módulos en VHDL y un procesador MIPS embebido. El sistema debe permitir el control de la serpiente a través de botones físicos, gestionar eventos del juego como colisiones y pausas, y mostrar el estado en tiempo real en una pantalla VGA.

El principal desafío consiste en coordinar de forma eficiente el flujo de datos entre el procesador MIPS (software) y los módulos gráficos (hardware), garantizando un tiempo de respuesta adecuado y un funcionamiento sincronizado del juego.

3.2 Entorno de desarrollo

El entorno utilizado para el desarrollo del sistema incluye:

- **Placa:** Zybo Z7-10 (Zynq-7000 SoC).
- **Lenguaje de diseño hardware:** VHDL.
- **Lenguaje de control:** Ensamblador MIPS.
- **Software de desarrollo:** Vivado Design Suite (Xilinx) para la síntesis del diseño digital y simulación.
- **Simulador:** Vivado Simulator y MARS para pruebas de código MIPS.
- **Visualización:** Interfaz VGA a 640x480 píxeles, representando una grilla lógica para el movimiento de la serpiente.

3.3 Arquitectura general del sistema

El sistema está compuesto por dos grandes bloques funcionales:

1. **Procesador MIPS:** Se encarga de leer entradas del usuario, calcular la lógica del juego (movimiento, crecimiento, colisiones) y comunicarse con los módulos VGA a través de registros mapeados.
2. **Módulo VGA en VHDL:** Encargado de generar la señal de video en tiempo real y manejar la visualización de los elementos del juego (cabeza, cola, fruta), además de detectar colisiones y generar flags de eventos.

3.4 Análisis funcional

Cada componente tiene una función crítica en el sistema:

- El software en MIPS debe mantenerse sincronizado con la señal de `tick` generada por hardware, para garantizar una velocidad constante del juego.
- El módulo VGA debe actualizar la imagen en pantalla en función del estado del juego y manejar correctamente la memoria de posiciones de la cola.
- El sistema debe responder en tiempo real a los botones del usuario, sin latencias visibles.
- Se deben evitar errores de escritura simultánea sobre los registros compartidos entre hardware y software.

3.5 Riesgos técnicos identificados

Durante el análisis se identificaron los siguientes riesgos técnicos:

- Pérdida de sincronización entre el software MIPS y el hardware VGA.
- Latencia en la detección de colisiones o respuesta al usuario.
- Problemas de visualización por mal direccionamiento de píxeles.
- Fallas en la generación aleatoria de posiciones para la fruta si no se utiliza un mecanismo lo suficientemente variado.

4 Capítulo 3: Diseño

4.1 Diagrama de bloques del sistema

El sistema completo está dividido en dos subsistemas principales:

- **Software de control (MIPS):** ejecuta la lógica del juego y se comunica con el hardware mediante registros mapeados.
- **Hardware de visualización y eventos (VHDL):** genera la señal VGA, dibuja la serpiente y la fruta, y detecta eventos como colisiones.

Los dos bloques están interconectados mediante un bus interno que permite la lectura y escritura de registros compartidos, con direcciones predefinidas para cada componente.

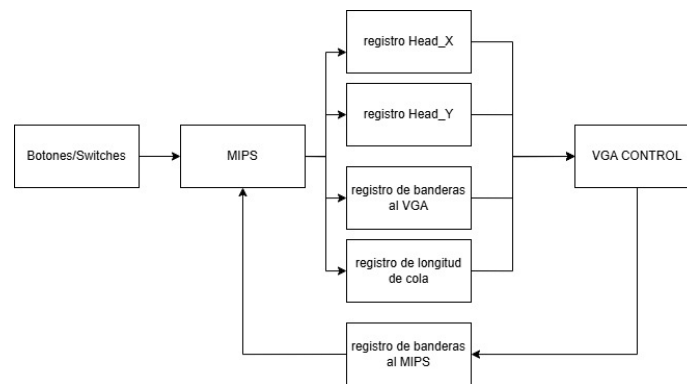


Figure 2: Diagrama de bloques entre el MIPS y el VGA

4.2 Módulos VHDL diseñados

El sistema cuenta con varios módulos desarrollados en VHDL que trabajan en conjunto para generar la visualización del juego, gestionar el movimiento de la serpiente y detectar los eventos clave. A continuación, se describen los principales:

- **snake_vga_display.vhd:** Módulo encargado de renderizar gráficamente la cabeza, la cola y la fruta en la salida VGA. Utiliza una grilla lógica que divide

la resolución de pantalla en bloques (por ejemplo, 62x46), donde cada bloque representa una unidad de juego. La visualización se realiza comparando coordenadas X e Y con las posiciones actuales de la serpiente y la fruta.

- **vga_sync.vhd:** Módulo que genera las señales de sincronización horizontal (**hsync**) y vertical (**vsync**) requeridas por el protocolo VGA (640x480 a 60 Hz). También provee las coordenadas actuales del barrido de píxeles en pantalla.
- **vga_demo_snake_btn.vhd:** Es el módulo principal que integra la lógica de juego en hardware. Se encarga de:
 - Mover internamente la cola en función de la dirección y posición de la cabeza recibidas desde el software MIPS.
 - Detectar colisiones con la fruta y activar la bandera correspondiente (**colision_fruta**).
 - Detectar autocolisiones (cabeza contra cola), y activar la bandera de reinicio (**colision_detectada**).
 - Coordinar la activación y consumo de frutas, mediante la gestión de señales como **fruta_generada**.
 - Gestionar el crecimiento de la cola en hardware cuando se detecta una colisión con fruta.
- **random.vhd:** Este módulo genera posiciones pseudoaleatorias para la fruta. Utiliza dos contadores caóticos (uno para eje X y otro para eje Y) que se actualizan continuamente, pero su valor se captura solo en momentos determinados por eventos como la colisión con fruta. Este mecanismo permite una pseudo-randomización sin necesidad de un generador aleatorio complejo. Las nuevas coordenadas generadas se envían a "vga_demo_snake_btn.vhd" para actualizar la posición de la fruta.

4.3 Programa en MIPS

El código ensamblador implementado en el procesador MIPS realiza las siguientes tareas:

- Lectura de entradas de usuario desde un registro mapeado que almacena el estado de los botones y switches.
- Priorización de señales de control: primero **reset**, luego **pausa**, y finalmente las direcciones.

- Cálculo de la nueva posición de la cabeza de la serpiente.
- Escritura de la nueva posición y dirección en los registros compartidos.
- Lectura de flags generados por el hardware para:
 - Aumentar la longitud de la cola al comer una fruta.
 - Reiniciar el juego en caso de colisión con la cola.

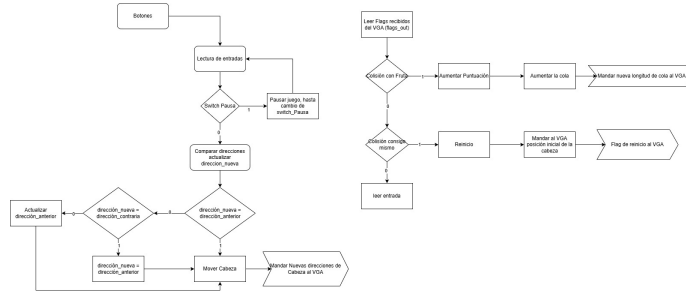


Figure 3: Diagrama de bloques del MIPS

4.4 Comunicación entre hardware y software

La interfaz entre MIPS y los módulos en VHDL se basa en un esquema de memoria mapeada. Cada componente clave tiene asignada una dirección fija que el software puede leer o escribir.

- `VGA_HEAD_X_ADDR`, `VGA_HEAD_Y_ADDR`: coordenadas de la cabeza.
- `VGA_COLA_LEN_ADDR`: longitud actual de la cola.
- `VGA_FLAGS_IN_ADDR`, `VGA_FLAGS_OUT_ADDR`: banderas de control y eventos.
- `TICK_ADDR`: sincronización de movimiento (espera activa).
- `VGA_FRUIT_ADDR`: coordenadas de la fruta generadas por hardware.

Todas estas direcciones se definen como constantes en el código ensamblador para facilitar su uso y mantener claridad en el acceso a los periféricos.

5 Capítulo 4: Implementación

5.1 Flujo general de ejecución

La implementación del sistema se basa en la interacción constante entre el procesador MIPS (software) y los módulos VHDL (hardware). El software controla la lógica del juego (dirección, estado, crecimiento), mientras que el hardware se encarga de la visualización, el movimiento visual de la cola, y la detección de eventos como colisiones o consumo de frutas.

El flujo de ejecución sigue el siguiente ciclo:

1. El software MIPS lee las entradas del usuario (botones y switches).
2. Se determina la nueva dirección de movimiento.
3. Se actualizan los registros mapeados: posición de la cabeza, longitud de la cola, flags de control.
4. El módulo VHDL detecta eventos (colisión con fruta o con la cola) y activa banderas.
5. El software interpreta esas banderas y actúa en consecuencia:
 - Si hay colisión con fruta: incrementa la longitud de la cola y solicita una nueva fruta.
 - Si hay colisión con la cola: reinicia el juego.

5.2 Conexión entre módulos

La conexión entre el MIPS y los bloques de VHDL se realiza a través de registros mapeados en memoria. Cada módulo accede o expone datos mediante direcciones definidas en el espacio de memoria del procesador.

Por ejemplo:

- 0xFFFFC000: Posición X de la cabeza.
- 0xFFFFC001: Posición Y de la cabeza.
- 0xFFFFC002: Longitud de la cola.
- 0xFFFFC003: Flags de entrada (pausa, reset, fruta_ok, pedir_fruta).

- `0xFFFFC005`: Flags de salida (colisión con cola, fruta lista).

Estas direcciones son definidas como constantes en el programa MIPS mediante la directiva `‘.eqv’`.

5.3 Código MIPS y control del juego

El programa en ensamblador MIPS sigue una estructura sencilla:

- En un bucle principal, espera una señal de `tick` (que puede estar sincronizada con el barrido VGA).
- Verifica el estado de los switches. Si está activo el `reset`, reinicia todas las variables y estados del juego.
- Si no hay reset, verifica si el switch de `pausa` está activo. En ese caso, congela la lógica de movimiento.
- Si el juego está activo, lee los botones y actualiza la dirección.
- Calcula la nueva posición de la cabeza, considerando posibles teletransportes en bordes.
- Escribe las coordenadas actualizadas y la longitud de la cola.
- Lee las flags de colisión:
 - Si hay colisión con fruta, incrementa `cola_len` y activa la señal `pedir_fruta`.
 - Si hay colisión con la cola, activa el reinicio del juego.

5.4 Generación de fruta y lógica caótica

La generación de nuevas frutas está completamente manejada por el hardware. Cuando el software activa la señal `pedir_fruta`, el módulo `‘random.vhd’` captura el valor de dos contadores caóticos (uno para X y otro para Y), que se actualizan en paralelo a eventos del usuario, garantizando una buena dispersión de posiciones.

El valor generado es devuelto al software a través de la señal `fruta_ok` y la nueva posición se refleja en la pantalla mediante la actualización del registro `VGA_FRUIT_ADDR`.

Esta solución evita el uso de un generador aleatorio complejo y reduce la carga del software, delegando al hardware una parte importante de la lógica del juego.

6 Capítulo 5: Pruebas y Simulación

6.1 Pruebas en simulación VHDL

Durante la etapa de desarrollo se realizaron simulaciones de los módulos principales del sistema utilizando el entorno de simulación de Vivado. Estas simulaciones permitieron validar el funcionamiento interno de los bloques antes de su integración con el procesador MIPS y la salida VGA.

Los módulos simulados incluyen:

- **vga_sync.vhd:** Se verificaron los tiempos de sincronización horizontal y vertical, comprobando que cumplen con los valores esperados para una resolución de 640x480 a 60 Hz.
- **snake_vga_display.vhd:** Se simuló el correcto mapeo de coordenadas (X, Y) a píxeles visibles en la pantalla. Se comprobó que la cabeza, la cola y la fruta se visualizan en las posiciones correctas dentro de la grilla lógica.
- **vga_demo_snake_btn.vhd:** Se verificó la lógica de movimiento de la cola, la detección de colisión con fruta y con la cola, así como la activación de las banderas correspondientes (`colision_fruta`, `colision_detectada`).
- **random.vhd:** Se probó que los contadores caóticos generen diferentes valores en distintos momentos, garantizando una distribución variable de frutas.

Estas pruebas se realizaron en entorno de simulación funcional (no temporizado), pero fueron fundamentales para corregir errores lógicos antes de cargar el diseño a la placa.

6.2 Pruebas funcionales en hardware

Una vez completada la síntesis del sistema y su implementación en la FPGA Zybo, se realizaron múltiples pruebas funcionales directamente sobre el hardware:

- **Control de dirección:** Se comprobó que los botones direccionales generan el movimiento esperado de la serpiente en todas las direcciones.
- **Detección de fruta:** Al colisionar con una fruta, se verificó que la cola crece correctamente y se genera una nueva fruta en otra posición.

- **Autocolisión:** Se probó que, al chocar con su propia cola, la serpiente se reinicia automáticamente.
- **Pausa y reinicio:** Se verificó el correcto funcionamiento del switch de pausa (detiene el movimiento), sin embargo el de reset (no se logró implementar el reinicio por switch).
- **Visualización VGA:** La señal VGA fue conectada a un monitor externo, observándose una imagen clara y estable, sin parpadeos, con representación correcta de todos los elementos del juego.

6.3 Análisis de resultados

El sistema se comportó de manera robusta en condiciones reales. Las pruebas demostraron:

- Baja latencia entre la entrada del usuario y la reacción del juego.
- Precisión en la detección de eventos y en la actualización gráfica.
- Estabilidad de la señal VGA a lo largo del tiempo.
- Generación efectiva de posiciones pseudoaleatorias para la fruta.

Además, la separación entre lógica de juego (software) y lógica de visualización (hardware) permitió un diseño modular y fácil de depurar.

6.4 Limitaciones y posibles mejoras

A pesar del buen funcionamiento general, se identificaron algunas limitaciones:

- La velocidad del juego depende del `tick` generado por polling desde el software; podría mejorarse mediante un temporizador hardware.
- Implementación del switch reset, para reinicio del juego.
- Actualmente el juego no incluye puntuación visible ni niveles de dificultad; se podrían agregar para mejorar la experiencia del usuario.
- No se implementó un módulo dedicado de detección de bordes; la teletransportación está codificado directamente en software, se podría implementar más modos de juego.

7 Conclusión

El desarrollo de este trabajo práctico permitió implementar un sistema embebido completo que integra software y hardware de manera coherente, funcional y estable. Se logró recrear exitosamente el clásico juego *Snake*, utilizando un procesador MIPS programado en lenguaje ensamblador y módulos en VHDL que gestionan la visualización y la detección de eventos mediante la salida VGA.

A lo largo del proyecto se pusieron en práctica múltiples conceptos vistos en la materia Microprocesadores 2, como la lectura de entradas mediante memoria mapeada, la escritura en periféricos, el uso de flags para sincronizar eventos, y la integración entre código ensamblador y lógica digital en FPGA. Además, se trabajó con señales sincronizadas, control de video en tiempo real y generación seudorandomizada de eventos en hardware, lo que dio como resultado un sistema interactivo, dinámico y entretenido.

Uno de los principales logros del proyecto fue la correcta comunicación entre el procesador MIPS y los módulos VGA mediante registros, permitiendo una separación clara de responsabilidades: el software toma decisiones de alto nivel mientras que el hardware ejecuta tareas repetitivas de forma eficiente y paralela.

También se destaca el uso creativo del módulo `random.vhd`, basado en contadores caóticos, para generar posiciones aleatorias de la fruta sin necesidad de un generador pseudoaleatorio clásico, lo cual demostró una solución ingeniosa y eficiente en recursos.

Durante la implementación se identificaron ciertas limitaciones y oportunidades de mejora, como la posible inclusión de un temporizador hardware para regular el `tick`, la implementación de un sistema de puntuación visible en pantalla y la mejora del mecanismo de generación aleatoria.

En términos generales, el trabajo práctico cumplió con los objetivos propuestos, funcionando correctamente en la FPGA Zybo y ofreciendo una experiencia fluida. Constituyó una experiencia valiosa para el afianzamiento de conocimientos de sistemas embebidos, diseño digital y programación de bajo nivel, además de fomentar la resolución de problemas de integración y depuración en sistemas mixtos.

Referencias

- [1] Patterson, D. A., y Hennessy, J. L., *Computer Organization and Design: The Hardware/Software Interface*. 4ta edición. Elsevier, 2009. [MIPS Green Card – resumen de instrucciones].
- [2] Digilent Inc., *ZYBO Zynq-7000 ARM/FPGA SoC Trainer Board Reference Manual*, Versión de febrero de 2017. Disponible en: <https://digilent.com/reference/programmable-logic/zybo>
- [3] Salah Eddin, Anas. *M14 - 6 - VGA - pixel clock rate*. Video disponible en YouTube: <https://www.youtube.com/watch?v=W1PrPJs7K-I>

8 Anexos

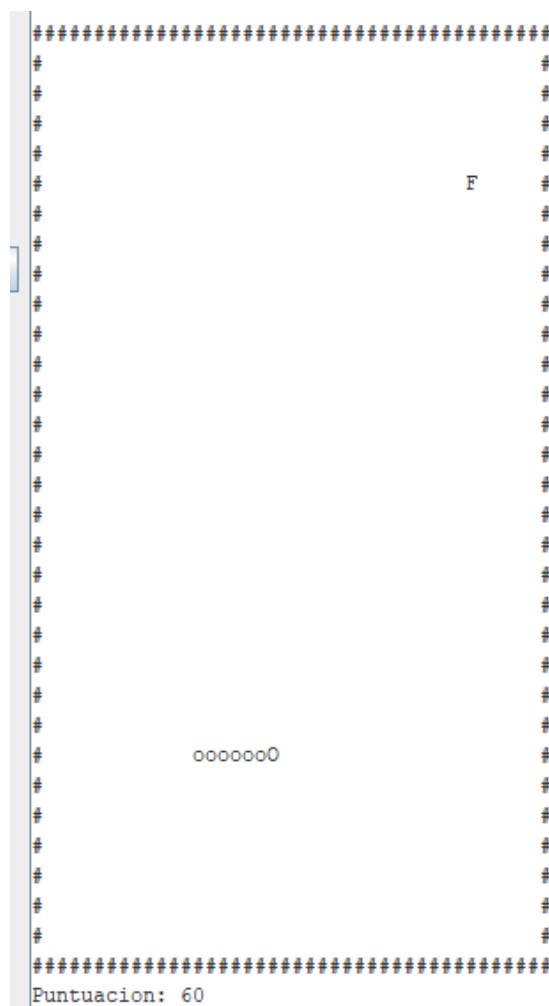


Figure 4: Snake inicial hecho en el MARS

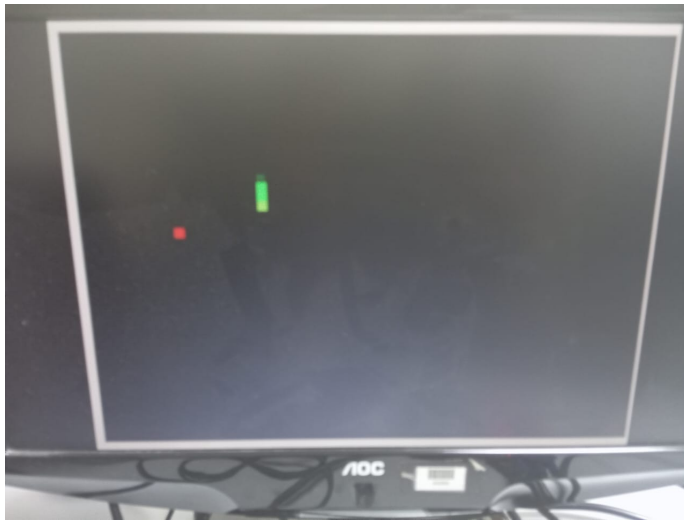


Figure 5: Snake final, implementación en el Zybo, visualización en VGA